



# Northern University

## Of Business and Technology Khulna

LAB Report - 02

Course Code: CSE 1202

Course Title: Object Oriented Programming Lab

**Submitted by:**

Name : Nazmus Sakib

Section: 2D

ID : 11220320888

Department: Computer Science and  
Engineering

**Submitted to:**

Name : Faria Afrin Tisha

Designation: Lecturer

Department: Computer Science and  
Engineering

Date of Submission: 22.02.2026

**Experiment No: 02****Experiment Name:** Identification of Const-Correctness Error in a C++ Class Program**Introduction**

In C++, const-correctness is an important principle that ensures objects declared as constant cannot modify their internal state. Violating this rule leads to compilation errors.

The given program defines a Point class with private data members and a member function that modifies those members. However, an error arises due to incorrect usage of the const keyword while creating an object.

**Input:**

```
#include <iostream>
using namespace std;

class Point
{
private:
    int x, y;

public:
    Point(int u, int v) : x(u), y(v) {}

    int getX() { return x; }
    int getY() { return y; }

    void doubleVal()
    {
        x *= 2;
        y *= 2;
    }
};

int main()
{
    const Point myPoint(5, 3);
    myPoint.doubleVal();
    cout << myPoint.getX() << " " << myPoint.getY() << endl;

    return 0;
```

```
}
```

**Output:**

Compilation Error:

A constant object cannot call a non-const member function that modifies its data members.

**Bug Report:**

The object myPoint is declared as const, which prevents any modification to its internal data. However, the member function doubleVal() modifies the values of x and y. Since a constant object cannot invoke a modifying function, the program results in a compilation error.

The issue can be resolved by removing the const qualifier from the object declaration.

**Discussion and Conclusion**

The error occurred due to improper use of const-correctness in C++. A constant object must not alter its internal state, and therefore cannot call non-const member functions that perform modifications.

By identifying and correcting the misuse of the const keyword, the program executes successfully. This experiment reinforces the importance of understanding object mutability and const behavior in C++ and strengthens debugging and analytical skills in object-oriented programming.